Process and Process APIs

READ, RECALL, THEN PRACTISE

StudyNotes original PDF pages **13-22**. Notes begin on page 2 of this pack. Original content and annotations are preserved.

- Draw process states and label each transition.

- Explain PCB, limited direct execution and context switching.

- Trace fork, wait and exec with a process tree.

SELECTED PRACTICE

Y15-A: APIs and context switching / original PDF pages 3, 7

WR: process explanations / original PDF pages 1, 2, 3

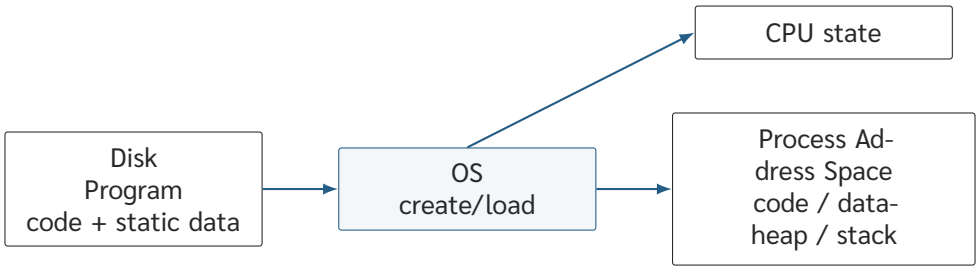
Each practice section has a source page and question guide. Mixed-topic source pages may appear in more than one pack.

Lecture 2 — Process and Process APIs

อ้างอิง: Lecture 2 หน้า 1–44 | process, สถานะและ PCB, context switch, timer interrupt และ process APIs

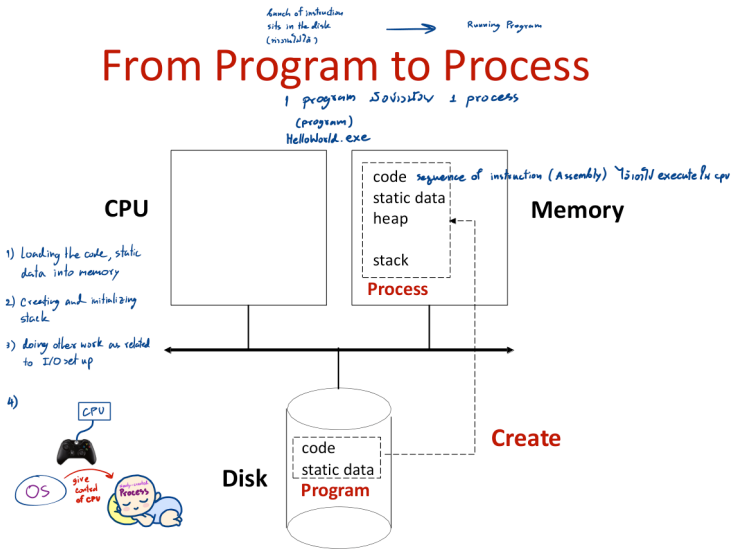
2.1 From Program to Process

> **เนื้อหาจาก Lecture**
>
> **Program** คือชุด instruction/code และ static data ที่เก็บอยู่บน disk ส่วน **Process** คือ program ที่กำลังถูก execute พร้อม execution state ของมัน การเปลี่ยนจาก program ไปเป็น process อย่างน้อยต้องมีการโหลด code/static data เข้า memory, สร้างและ initialize stack, จัดเตรียม state ที่เกี่ยวข้องกับ I/O/ runtime และทำให้ CPU สามารถเริ่ม execute ได้



ภาพประกอบจากสไลด์หน้า 3 ช่วยแยกความหมายของ program กับ process ให้ชัดเจน

ภาพประกอบ 2.1 Lecture 2 หน้า 3 — From Program to Process



จุดสังเกต

- ซ้ายล่าง: program อยู่บน disk เป็น code/static data ที่ยังไม่ execute
- OS สร้าง process โดย load code/data, initialize stack และ setup execution/I/O state
- กรอบ memory แสดง process image ที่มี code, static data, heap, stack

ที่มา: Lecture 2 หน้า 3 พร้อมหมายเหตุเรื่องการโหลดและ process

Program คือชุดคำสั่งที่จัดเก็บอยู่บน storage เมื่อ OS สร้าง process และเตรียม state ที่จำเป็นแล้ว CPU จึงเริ่มนำคำสั่งนั้นมาทำงานได้

2.2 Process = (P, D, C, S)

Lecture กำหนด process เป็น entity ที่มีสี่องค์ประกอบ:

Symbol	Component	ความหมายตาม Lecture
P	Program / code	code หรือชิ้นส่วน program ที่ process ผูกอยู่
D	Processing data	ชุดตัวแปรข้อมูลของ process
C	Control state	control variables ที่บอกว่าจะ execute ที่ไหน/อย่างไรต่อ
S	Process state	สถานะ เช่น Ready, Running, Blocked/Suspended

> **อธิบายให้เห็นภาพ**
> ถ้ามีเพียง code อย่างเดียว เรายัง resume process ไม่ได้ เพราะ CPU ต้องรู้ตำแหน่ง instruction ปัจจุบัน, register values, stack และสถานะการรอ resource ด้วย ดังนั้น process เป็นทั้งโปรแกรม **และ state ที่ทำให้การ execute ต่อเนื่องได้**

2.3 Process Address Space และ Process Descriptor

Lecture แบ่ง memory ที่เกี่ยวข้องกับ process เป็น:

1. **Code/Text Segment** — instructions
2. **Data Segment / Heap** — dynamic data เช่นสิ่งที่ได้จาก `malloc()`
3. **Stack** — parameters, local variables, return address และ state ของ function calls
4. **Process Descriptor / Process Control Block (PCB)** — เก็บใน kernel space และเข้าถึงได้ใน kernel mode

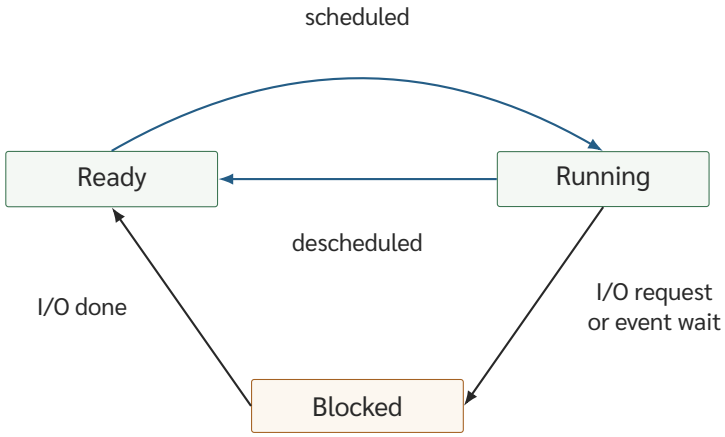


2.3.1 PCB เก็บอะไร

Lecture ระบุว่า PCB เป็น OS data structure สำหรับข้อมูลที่สัมพันธ์กับแต่ละ process เช่น identifier, state, CPU context/register state, scheduling/accounting information และข้อมูล resource ที่จำเป็นต่อการจัดการ process

> **เชื่อมกับ Computer Architecture**
> PCB สำคัญเพราะ CPU มี state ที่เปลี่ยนตลอดเวลา ได้แก่ **Program Counter (PC)**, general-purpose registers, stack pointer และ processor control state เนื้อหา Computer Architecture Lecture 7 แสดงให้เห็นว่า instruction execution อาศัย state เหล่านี้ เมื่อ OS หยุด Process A ไว้ชั่วคราวและจะกลับมาทำงานต่อ จึงต้องบันทึกค่า state ไว้ใน PCB

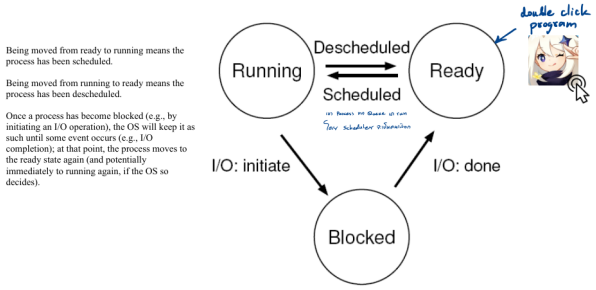
2.4 Process State: Ready, Running, Blocked



- **Ready**: พร้อมรัน แต่ยังไม่ได้ CPU
- **Running**: กำลัง execute บน processor
- **Blocked**: ทำ operation บางอย่างแล้วไม่พร้อมรัน เช่นกำลังรอ I/O completion

หมายเหตุในสไลด์หน้า 6 กำกับ transition ไว้ว่า เมื่อเลือก process จาก Ready ให้ใช้ CPU จะเรียกว่า Scheduled และเมื่อถอด process ออกจาก CPU จะเรียกว่า Descheduled

ภาพประกอบ 2.2 Lecture 2 หน้า 6 — Ready / Running / Blocked



จุดสังเกต

- **Ready**: พร้อมใช้ CPU แต่ยังไม่ถูกเลือก
- **Running**: กำลัง execute instruction
- **Blocked**: ยังรันต่อไม่ได้จน event/I/O เสร็จ
- Scheduled/Descheduled เป็น transition ระหว่าง Ready↔Running; I/O initiate/done เชื่อม Running↔Blocked↔Ready

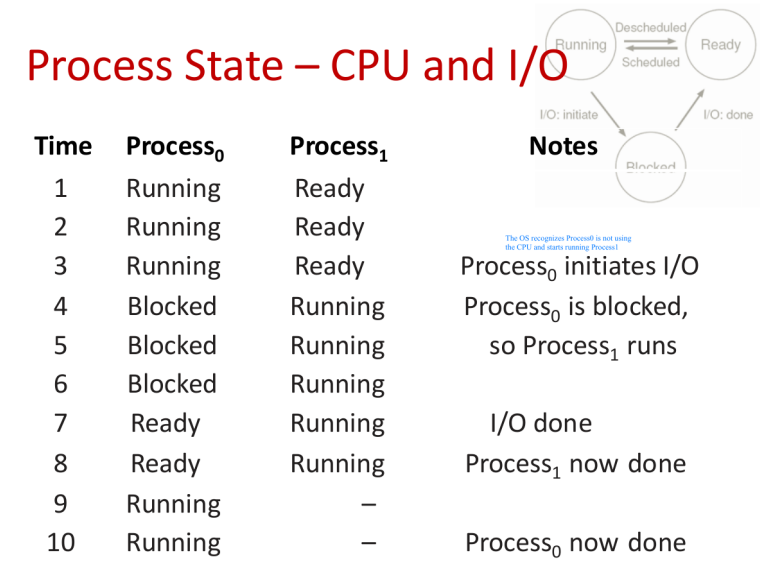
ที่มา: Lecture 2 หน้า 6 พร้อมหมายเหตุเรื่องสถานะ process

2.4.1 CPU-only vs CPU + I/O timeline

สไลด์หน้า 7–8 แสดงว่า process ที่ใช้ CPU อย่างเดียวสามารถอยู่ Running ต่อเนื่องจนถูกสลับ ส่วน process ที่ออก I/O จะเปลี่ยนเป็น Blocked ทำให้ OS มีโอกาสให้ process อื่นใช้ CPU ระหว่างรอ

> **จุดที่มักสับสน**
>
> **Ready ต่างจาก Blocked.** ทั้งคู่ ไม่ได้ใช้ CPU ตอนนี้ เหมือนกัน แต่ Ready สามารถถูก scheduler เลือกได้ทันที ส่วน Blocked ยังมีเงื่อนไขที่ต้องเสร็จก่อน (เช่น I/O done)

ภาพประกอบ 2.3 Lecture 2 หน้า 8 — CPU + I/O Process Timeline



Time	$Process_0$	$Process_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	$Process_0$ initiates I/O
4	Blocked	Running	$Process_0$ is blocked,
5	Blocked	Running	so $Process_1$ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	$Process_1$ now done
9	Running	–	
10	Running	–	$Process_0$ now done

จุดสังเกต

- เมื่อ Process 0 initiate I/O จะออกจาก Running ไป Blocked
- CPU จึงไม่จำเป็นต้อง idle: scheduler เลือก Process 1 จาก Ready มารันได้
- เมื่อ I/O done, Process 0 กลับ Ready ไม่ได้แปลว่าต้อง Running ทันที

ที่มา: Lecture 2 หน้า 8 ใช้เชื่อม state transition กับการใช้ CPU

2.5 Typical Process APIs และ OS Data Structures

Lecture ยก operation หลัก ๆ ที่ OS ต้องรองรับ: create, destroy/terminate, wait, miscellaneous control, status query และ communication. API เหล่านี้ทำให้ user process ขอให้ OS จัดการ lifecycle ของ process โดยไม่ต้องแก้ kernel structures เอง

OS ต้องมี process lists/queues และ PCB เพื่อ track ว่า process ใด Ready, Running, Blocked และมี resource/state อะไรบ้าง

2.6 CPU Virtualization Challenge: Performance + Control

จุดเปลี่ยนของ Lecture คือคำถามว่า OS จะให้ process รัน เกือบตรงบน CPU เพื่อ performance แต่ยังรักษา control ได้อย่างไร

> **เนื้อหาจาก Lecture**
>
> Limited Direct Execution (LDE) แก้โจทย์โดยประกอบด้วยสามส่วนตามสไลด์:
> 1. Restricted Operation
> 2. Switch Between Processes / Context Switch
> 3. Timer Interrupt

2.6.1 1) Restricted Operation — User Mode / Kernel Mode

Kernel mode สามารถทำ privileged operations ได้ ส่วน user mode ถูกจำกัด ตัวอย่างใน Lecture: user process ไม่สามารถขอ I/O โดยตรง; หากต้องเข้าถึง disk ต้องผ่าน system call

ขั้นตอนสำคัญ:

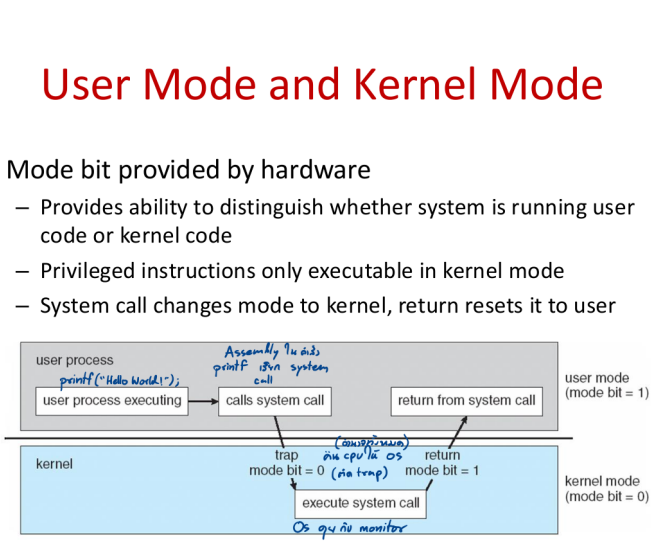
1. user program execute special instruction **trap**
2. privilege ถูกยกไป kernel mode
3. kernel ทำ privileged operation
4. kernel ใช้ **return-from-trap** เพื่อกลับ user code และลด privilege

2.6.2 Trap Table

OS บอก hardware ว่าเมื่อ exceptional event แต่ละชนิดเกิดขึ้นควรวิ่งไป code ใดผ่าน **trap table** ซึ่งถูกตั้งค่าตอน boot. Lecture ยก hard-disk interrupt เป็นตัวอย่างหนึ่งของ event ที่ต้องมี handler address ที่เชื่อถือได้

> **จุดที่มักสับสน**
>
> Trap table ต้องถูกตั้งโดย kernel/privileged code. ถ้า user process เปลี่ยน trap table เองได้ มันจะเปลี่ยน control flow ของ privileged event และทำลาย protection model ได้

ภาพประกอบ 2.4 Lecture 2 หน้า 19 — Hardware Mode Bit



จุดสังเกต

- Hardware mode bit เป็นส่วนของ mechanism ที่แยก privileged execution ออกจาก user execution
- trap ทำให้ control เข้า kernel ผ่าน entry ที่ตั้งไว้; return กลับ user พร้อมลด privilege
- นี่คือฐานของ **Limited Direct Execution**: รัน user code ตรงบน CPU แต่จำกัดสิ่งอันตราย

ที่มา: Lecture 2 หน้า 19.

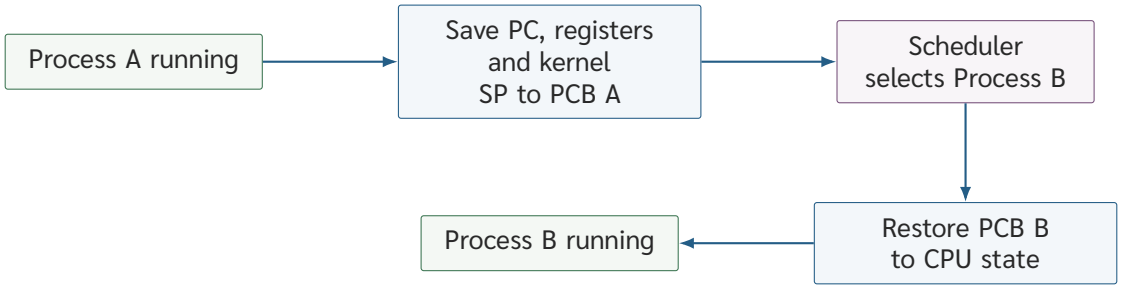
2.7 2) Switch Between Processes — Scheduler vs Context Switch

Lecture แยกสองชั้น:

- **Scheduler**: ตัดสินใจว่า **ควร switch ไป process ไหน**
- **Context Switch**: low-level code ที่ save/restore state เพื่อทำ switch จริง

> **เนื้อหาจาก Lecture**
>
> **Context Switch** คือขั้นตอน storing และ restoring state ของ process เพื่อให้ภายหลังกลับมา execute ต่อจากจุดเดิมได้

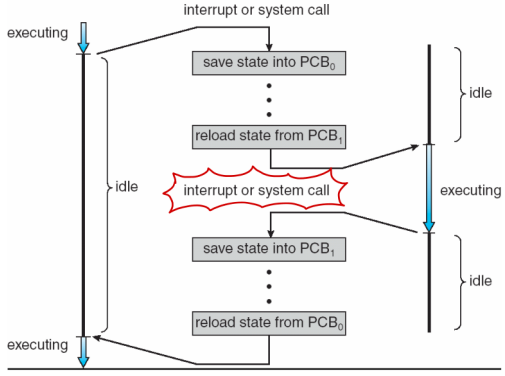
ตาม Lecture OS save general-purpose registers, PC และ kernel stack pointer ลง state/PCB ของ process ปัจจุบันแล้ว restore context ของ process ถัดไปจาก Ready queue; เมื่อ execute return-from-trap, process ใหม่จึงกลับเข้าการ execute ที่ state ของมัน



> **เชื่อมกับ Computer Architecture**
>
> Architecture Lecture 7 แสดงว่า PC บอก instruction sequence, register input/output ถูกควบคุมด้วย control signals และ memory access ผ่าน MAR/MDR. Context switch จึงไม่ใช่แค่ เปลี่ยนชื่อ process แต่เป็นการเปลี่ยน processor-visible state. หาก restore PC/register ผิดแม้ค่าเดียว process จะ resume ผิดจุดหรือใช้ข้อมูลของอีก process

Lecture ให้ intuition เรื่อง cost ว่า context switch อยู่ระดับ sub-microsecond บนระบบ 2–3 GHz ตามค่าที่สไลด์ยกมา; ประเด็นสำคัญคือ switch มี overhead จึงไม่ควรถือว่าฟรี

ภาพประกอบ 2.5 Lecture 2 หน้า 23 — Saving / Restoring Context



จุดสังเกต

- Context ที่ต้องรักษาเกี่ยวกับ PC, registers และ kernel stack/state ที่ทำให้ process resume ถูกจุด
- ฝั่งออกจาก A = save; ฝั่งเข้า B = restore
- Scheduler เลือก process; Context Switch ทำการเปลี่ยน CPU state — สองคำนี้ไม่ใช่สิ่งเดียวกัน

ที่มา: Lecture 2 หน้า 23 เชื่อมกับ CPU register state เพื่ออธิบายเหตุผลที่ต้อง save และ restore

2.8 3) OS จะเอา CPU กลับมาได้อย่างไร

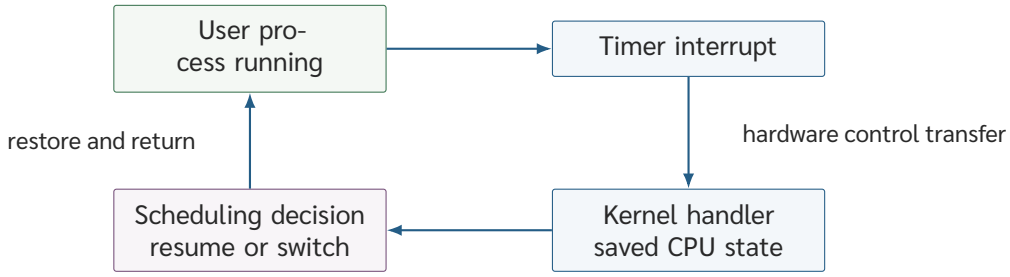
2.8.1 การคืน CPU แบบ Cooperative

วิธีนี้อาศัยให้ process คืน CPU ผ่าน system call, การเรียก `yield()` หรือเหตุการณ์ที่ทำให้ trap เข้า OS ข้อจำกัดคือหาก process ติดอยู่ใน infinite loop และไม่เรียก yield หรือ system call OS อาจดึง CPU กลับมาไม่ได้ Lecture จึงยกกรณีร้ายแรงที่ต้อง reboot ระบบ

2.8.2 การดึง CPU กลับโดยไม่พึ่ง Process: Timer Interrupt

OS ตั้ง timer device ให้ส่ง interrupt เป็นระยะ เมื่อเกิด interrupt จะมีลำดับดังนี้:

1. process ที่กำลังรันถูกหยุดชั่วคราว
2. hardware บันทึก state ที่จำเป็นต่อการกลับมาทำงานต่อ
3. interrupt handler ที่ OS เตรียมไว้ได้รับ control
4. OS ตัดสินใจว่าจะให้ process เดิมทำงานต่อหรือสลับไป process อื่น

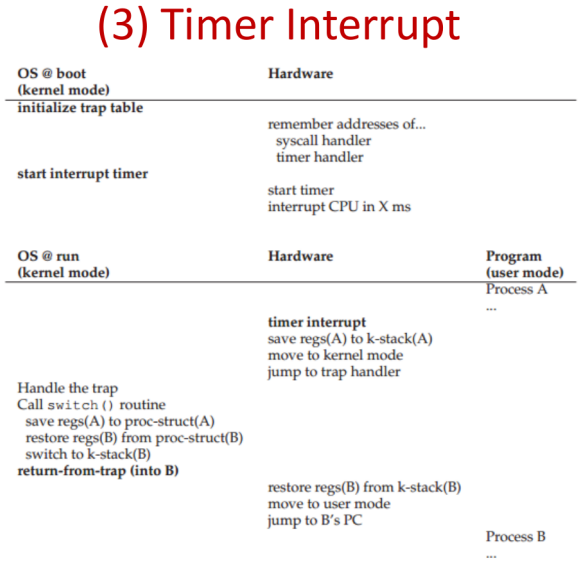


> **เชื่อมกับ Computer Architecture**
>
> Computer Architecture Lecture 5 อธิบาย hardware interrupt, IRQ, ISR, การเปิด/ปิด interrupt และ return-from-interrupt นอกจากนี้ยังชี้ว่า interrupt latency รวมเวลาที่ใช้บันทึกและคืน state กลไก **timer interrupt** จึงทำให้ kernel ดึง control กลับมาได้โดยไม่ต้องรอให้ user process ยินยอม

> **จุดเน้นก่อนสอบ**
>
> หัวใจของ LDE ตามสไลด์หน้า 30 คือ **Restricted Operation, Context Switch และ Timer Interrupt** ควรแยกให้ได้ว่าแต่ละกลไกทำหน้าที่ป้องกันการเข้าถึง สลับ state และดึง control กลับมาอย่างไร

2.9 UNIX Process APIs

ภาพประกอบ 2.6 Lecture 2 หน้า 29 — Timer Interrupt



จุดสังเกต

- Timer interrupt ทำให้ OS ดึง CPU กลับมาได้โดยไม่ต้องรอให้ process yield
- เมื่อเข้า interrupt handler แล้ว kernel จะตัดสินใจว่าจะ resume process เดิมหรือสลับ process
- ภาพนี้เชื่อม Mechanism (timer/context switch) กับ Policy (scheduler)

ที่มา: Lecture 2 หน้า 29 พร้อมลำดับที่เขียนกำกับไว้ในสไลด์

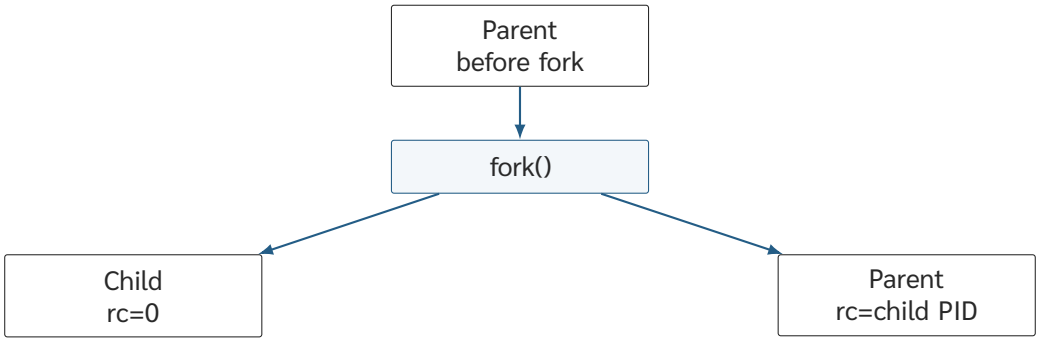
2.9.1 fork(): สร้าง process ใหม่

Lecture ใช้ code ต่อไปนี้เพื่อแสดง parent/child path:

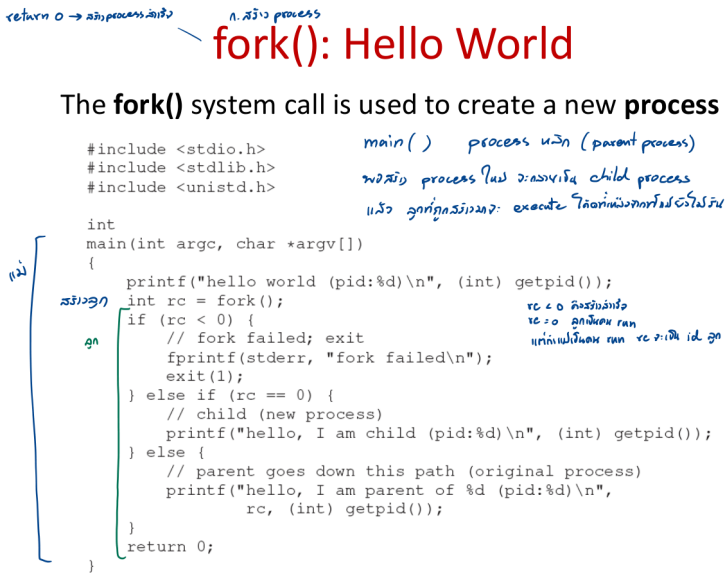
```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {
        // parent goes down this path (original process)
        printf("hello, I am parent of %d (pid:%d)\n",
               rc, (int) getpid());
    }
    return 0;
}
```

- child เป็น process ใหม่และเริ่มต่อจากจุดหลัง `fork()` ไม่ได้เริ่มใหม่ที่ `main()`
- child เป็น almost exact copy แต่มี address space, registers, PC ฯลฯ ของตนเอง
- ใน child: return value ของ `fork()` = 0
- ใน parent: return value คือ PID ของ child; กรณี error < 0
- บน single CPU ไม่มีการรับประกันว่า parent หรือ child จะได้ทำงานก่อน ลำดับ output จึงเปลี่ยนได้



ภาพประกอบ 2.7 Lecture 2 หน้า 34 — fork() Example + Annotation



จุดสังเกต

- fork สำเร็จแล้ว execution แตกเป็น parent และ child จาก instruction หลัง fork
- child เห็น return value = 0; parent เห็น child PID
- output ordering ยังไม่ deterministic ถ้าไม่มี wait/synchronization

ที่มา: Lecture 2 หน้า 34 พร้อมหมายเหตุเรื่อง return value ของ parent และ child

2.9.2 wait() / waitpid(): จัดลำดับ parent–child

`wait()` ทำให้ parent หยุดรอจนกว่า child ตัวหนึ่งจะ terminate ส่วน `waitpid()` ใช้รอ child ที่ระบุ ในตัวอย่างของ Lecture การเพิ่ม wait ทำให้ child แสดงผลก่อน parent อย่างแน่นอน ทั้งสองฟังก์ชันคืน PID และรับ exit status ผ่าน status argument

```
// parent path
int wc = wait(NULL);
printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
       rc, wc, (int) getpid());
```

2.9.3 exec(): เปลี่ยน program image ที่ process กำลังรัน

Lecture ระบุ family ของ `exec()` เช่น `execvp()`. ใช้เมื่อต้องการให้ child รัน program ที่ต่างจาก parent:

```
char *myargs[3];
myargs[0] = strdup("word_count");
myargs[1] = strdup("p.txt");
myargs[2] = NULL;
execvp(myargs[0], myargs);
printf("this shouldn't print out");
```

หาก `exec()` สำเร็จ มันไม่ return มายัง instruction ถัดไป เพราะ process image ถูกแทนด้วย executable ใหม่

> **จุดที่มักสับสน**
>
> **fork() ต่างจาก exec()** fork สร้าง process ใหม่ที่ทำงานต่อจากจุดเดิม ส่วน exec เปลี่ยน program image ของ process ที่มีอยู่ Unix shell แยกสอง operation นี้เพื่อให้ตั้งค่า environment หรือ redirection ได้ระหว่าง fork กับ exec

2.9.4 Termination, signals, command-line tools

Process จบการทำงานได้ด้วย `exit(status)` หรือจาก fatal error/signal จากนั้น OS คืน resources และ parent สามารถอ่าน exit status ผ่าน wait ได้ Lecture ยังกล่าวถึง `kill()` และ signal subsystem สำหรับควบคุม process รวมถึง `ps` และ `top` สำหรับตรวจดูสถานะ

2.9.5 Why fork() + exec() ช่วยสร้าง shell

ลำดับการทำงานตาม Lecture:

1. shell ค้นหา executable ที่ตรงกับ command
2. เรียก `fork()` เพื่อสร้าง child
3. ตั้งค่า environment หรือ redirect I/O ก่อนเรียก exec
4. child เรียกฟังก์ชันในกลุ่ม `exec()`
5. shell หรือ parent เรียก `wait()`
6. เมื่อ child ทำงานเสร็จ shell กลับมาแสดง prompt

ตัวอย่าง `wc p.txt > newfile.txt`: shell ปิด standard output แล้วเปิด `newfile.txt` ก่อนเรียก exec ทำให้ program ที่ถูก exec เขียนผลลงไฟล์ได้ โดยไม่ต้องรู้รายละเอียดการทำงานของ shell

ภาพประกอบ 2.8 Lecture 2 หน้า 42 — Why fork() and exec()?

Why fork() and exec()? (1)

- The separation of **fork()** and **exec()** is essential in building a Unix shell (command-line-interpreter) because it lets the shell run code after the call to **fork()** but before the call to **exec()**
- With the name of an executable program, plus any arguments passed to the Unix shell, it:
  - figures out where the executable is
  - calls **fork()** to create a new child process
  - alters the environment of the about-to-be-run program, if necessary
  - calls some variant of **exec()** to run the command
  - waits for the command to complete by calling **wait()**
  - when the child completes, the shell returns from the **wait()** and prints out a prompt again, ready for the next command

จุดสังเกต

- จุดสำคัญคือ **ช่องว่างระหว่าง fork กับ exec**: shell ยังมีโอกาส setup environment / file descriptors ก่อนเปลี่ยน program image
- ลำดับบนสไลด์เริ่มจากค้นหา executable จากนั้น fork และตั้งค่า environment ก่อน exec ส่วน parent จะ wait แล้วจึงกลับสู่ prompt
- นี่อธิบายว่าทำไม Unix แยก สร้าง process ออกจาก เปลี่ยน program ที่ process รัน

ที่มา: Lecture 2 หน้า 42.

2.9.6 Pipes

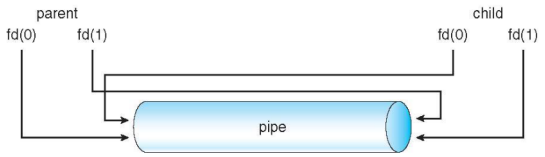
Lecture กำหนด pipe แบบ producer/consumer:

- producer เขียน write-end
- consumer อ่าน read-end
- `int pipe(int fd[2])`: `fd[0]` สำหรับ read, `fd[1]` สำหรับ write
- ถ้า parent รับข้อมูลจาก child: parent ปิด `fd[1]` แล้วอ่าน `fd[0]`; child ปิด `fd[0]` แล้วเขียน `fd[1]`

ภาพประกอบ 2.9 Lecture 2 หน้า 44 — Pipes

Pipes

- Pipes allow communication in a producer/consumer style
  - Producer writes to one end (the write-end of the pipe)
  - Consumer reads from the other end (the read-end of the pipe)
  - If the parent wants to receive data from the child, it should close *fd[1]* and read from *fd[0]*, and the child should close *fd[0]* and write to *fd[1]*



  - **int pipe(int** *fd[2]***);** *fd[0]* is set up for reading, *fd[1]* for writing

จุดสังเกต

- `fd[0]` คือ read end; `fd[1]` คือ write end
- หาก parent รับข้อมูลจาก child ให้แต่ละฝั่งปิด end ที่ไม่ได้ใช้ เพื่อให้ dataflow เป็นไปตามทิศทางที่ต้องการ
- มอง pipe เป็นช่องทาง producer/consumer ที่ kernel จัดการ ไม่ใช่ shared variable ทั่วไป

ที่มา: Lecture 2 หน้า 44 ทิศทางของ endpoint แสดงไว้ในภาพเพื่อให้อ่านตามได้ง่าย

2.10 Chapter Summary

สรุปท้ายบท

- Process คือ program ที่กำลังทำงานพร้อม data และ state ที่จำเป็น PCB เก็บข้อมูลฝั่ง kernel ที่ใช้จัดการและ resume process
- Process มี state หลักคือ Ready, Running และ Blocked เมื่อเริ่ม I/O จะเปลี่ยนจาก Running เป็น Blocked และกลับเป็น Ready เมื่อ I/O เสร็จ
- CPU virtualization ใช้ Limited Direct Execution: Restricted Operation + Context Switch + Timer Interrupt

- Scheduler เป็น policy; Context Switch เป็น mechanism
- Timer Interrupt แก้ปัญหา process ไม่ยอมคืน CPU ด้วยตัวเอง
- UNIX model แยก process creation (fork), program replacement (exec), synchronization (wait), termination/signals และ pipe communication

Key Mechanisms

User/Kernel mode, trap table, trap/return-from-trap, PCB save/restore, timer interrupt, context switch.

Key Policies

การเลือก process ถัดไปเป็นหน้าที่ของ scheduler policy ซึ่งจะเรียนต่อใน Lecture 3

ComArch Connections

PC/register/kernel stack pointer, interrupt handler, privilege, processor state, controlled return path.

Common Confusions

- Program ต่างจาก Process
- Ready ต่างจาก Blocked
- Scheduler decision ต่างจาก Context Switch implementation
- System Call trap ต่างจาก Timer Interrupt แม้ทั้งคู่พา control เข้า kernel
- Context Switch ต่างจาก Process Swapping ไป disk (จะพบใน Lecture 6)
- fork ต่างจาก exec

Quick Revision Checklist

☐ วาด Ready–Running–Blocked transition ได้
☐ อธิบาย PCB ว่าทำไมต้องมีและเกี่ยวกับ CPU state อย่างไร
☐ อธิบาย LDE ตั้งแต่ user execution ผ่าน trap หรือ timer interrupt เข้าสู่ kernel จน context switch และ return ได้
☐ อธิบาย cooperative vs non-cooperative CPU control ได้
☐ trace return values ของ fork และ output ordering ได้
☐ อธิบาย fork + exec + wait flow ของ shell ได้

Practice: Y15-A: APIs and context switching

ORIGINAL SOURCE

Y15-A / 2015 Operating Systems exam, annotated copy A (9 pages)

Full source filename is listed under this code in PRACTICE_INDEX.md.

Original PDF page(s): **3, 7**

Page 3: questions 5-7. Page 7: question III(a). Thread-specific questions are outside the requested Lecture 1-6 scope.

ANSWER ANNOTATIONS

Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

ชื่อ .....รหัสประจำตัว.....ภาควิชา/ชั้นปี.....

5. The address of the next instruction to be executed by the current process is provided by

a) CPU general purposed registers

b) Program counter

c) Process stack

d) Process pipe

e) Process context

f) Process heap

6. Which of the following system calls does not return control to the calling point, on termination

Ref: Operating System Three Easy Pieces
Topic - Interlude: API (Page 2)

Now the interesting part begins. The process calls the `fork()` system call, which the OS provides as a way to create a new process. The odd part: the process that is created is an (almost) *exact copy of the calling process*. That means that to the OS, it now looks like there are two copies of the program `p1` running, and both are about to return from the `fork()`

Lec 2

exec(): Family of Functions

- There is a family of **exec()** system calls, e.g. **execvp()**
- These are useful when you want to run a child program that is different from the parent program
- What it does: given the name of an executable (e.g. word_count), and some arguments (e.g. p.txt)

```
// child (new process)
char *myargs[3];
myargs[0] = strdup("word_count"); // program: "word_count"
myargs[1] = strdup("p.txt"); // argument: file to count
myargs[2] = NULL; // marks end of array
execvp(myargs[0], myargs); // runs the word count code
printf("this shouldn't print out");
```

- A successful call to an **exec()** function never returns

Lec 2

wait(): Example Usage

- Sometimes, it is quite useful for a parent to wait for a child process to finish before moving on
- Using **wait()** or **waitpid()** will force the parent to wait
- Let's modify the last printf() of the previous code:

```
// parent goes down this path (original process)
{
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
                rc, wc, (int) getpid());
}
```

- The parent calls **wait()** to delay its execution until the child finishes executing – when the child is done, **wait()** returns to the parent

a) fork

b) exec

c) wait

d) none of the above

7. The following C program :

```
main()
{
    fork();
    fork();
    printf("hello");
}
```

fork(); สร้าง 1 process

fork(); ✓ สร้าง 1 process

printf("hello"); ✓✓✓

How may time the word "hello" will be printed

a) Only once

b) Twice

c) Three times

d) Four times

e) Eight times

Ref: Operating System Three Easy Pieces
Topic - Interlude: API (Page 5)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int
main(int argc, char *argv[])
{
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {           // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) { // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
        char *myargs[3];
        myargs[0] = strdup("wc");   // program: "wc" (word count)
        myargs[1] = strdup("p3.c"); // argument: file to count
        myargs[2] = NULL;           // marks end of array
        execvp(myargs[0], myargs);  // runs word count
        printf("this shouldn't print out");
    } else {                    // parent goes down this path (main)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
               rc, wc, (int) getpid());
    }
    return 0;
}
```

Figure 5.3: **Calling `fork()`, `wait()`, And `exec()` (`p3.c`)**

The `fork()` system call is strange; its partner in crime, `exec()`, is not so normal either. What it does: given the name of an executable (e.g., `wc`), and some arguments (e.g., `p3.c`), it **loads** code (and static data) from that executable and overwrites its current code segment (and current static data) with it; the heap and stack and other parts of the memory space of the program are re-initialized. Then the OS simply runs that program, passing in any arguments as the `argv` of that process. Thus, it does *not* create a new process; rather, it transforms the currently running program (formerly `p3`) into a different running program (`wc`). After the `exec()` in the child, it is almost as if `p3.c` never ran; a successful call to `exec()` never returns.

8. If a thread invokes the exec system call, Which statement(s) is/are true

a) Only the *exec* executes as a separate process

b) The program specified in the parameter to *exec* will replace the entire process

c) The *exec* is ignored as it is invoked by a thread

d) None of the above is correct

ชื่อ .....รหัสประจำตัว.....ภาควิชา/ชั้นปี.....

III. (20 points)

a) (10 points) Describe the concept of "context switching" and provide step-by-step information of how it is performed by the operating system.

b) (10 points) What are the primary differences between calling a function and cooperatively switching to (and from) another thread at user level, in the same address space? Is one more expensive than the other? Why or why not?

Practice: WR: process explanations

ORIGINAL SOURCE

WR / Written question collection (17 pages)

Full source filename is listed under this code in PRACTICE_INDEX.md.

Original PDF page(s): **1, 2, 3**

Focus on questions 6, 8-10, 13, 15, 18 and 20. Skip thread, semaphore and deadlock questions.

ANSWER ANNOTATIONS

Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

1. What did Alan Turing was trying to solve when he built the first computer? (2 pts)

   Enigma Code

2. Will computer work without Operating System? Why? (2 pts)

   No, OS manage hardware resources and provide an environment for application to run.

3. What could be a result of non-preemptive process system? (2 pts)

   Process may not respond to high-priority task.

4. How can Operating System preempt a process from execution? (2 pts)

   Interupt process to switch task.

5. What is a trap in Operating System? What does it do? (4 pts)

   A trap is system call, provide OS level service.

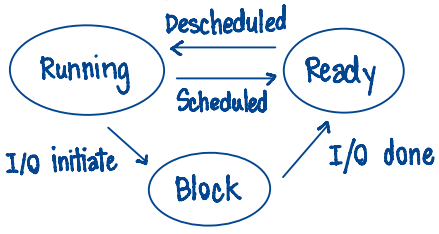
6. What is the different between a process and a program? (2 pts)

   A process is an execution of instruction inside a program, but a program is a set of instruction

7. What is the dual-mode operation of Operating System? (2 pts)

   User mode & kernel mode

8. Draw the state diagram of a process with events that cause the transition. (5 pts)



9. What are the steps performed in context switching? (2 pts)

1. Save the current process.
2. Load the state of the next process.
3. Update the CPU to execute the next process.

10. What is PCB? What kind of information is stored in PCB? (4 pts)

Process Control Block, it stores an information of process

process ID (PID)

11. What is a critical section? (2 pts)

12. What is the cause of a race condition? (2 pts)

13. What would happen if a parent process doesn't wait for return values from a child process? (2 pts)

The child process may become a zombie, and it exist until handled

14. What is the benefit of using thread over creating a new process? (2 pts)

Light weight, Reduce Overhead

15. What is an I/O bound process and a CPU bound process? (2 pts)

I/O bound process spend more time on I/O operation.

CPU bound process spend more time computation on using CPU.

16. How can you use a semaphore to create mutual exclusion? (2 pts)

17. What are the four conditions that lead to a deadlock? (4 pts)

18. What kind of event move process from running state to waiting queue? (2 pts)

Waiting

19. Why do you think there is no scheduling with longest job first? (2 pts)

It can cause high waiting time for shorter job

20. What is the command you use to create a process in Linux system? (2 pts)

fork()

21. What are the approaches for handling deadlock? (2 pts)

22. What does it mean when Banker Algorithm determine that the system is in an unsafe state? (2 pts)